

Lecture 6: Data Acquisition - APIs

James Sears*

AFRE 891/991 FS 25

Michigan State University

*Parts of these slides related to APIs are adapted from [“Data Science for Economists”](#) by Grant McDermott.

Table of Contents

1. **Prologue**
2. **APIs**
3. **Explicit API Endpoint, No Key**
4. **Explicit API Endpoint with Key**
5. **Hidden APIs**
6. **Considerate Web Scraping**

Prologue

Client-Side

Last time, we learned workflows for scraping **server-side** and **client-side** websites.

- Static sites with **rvest**
- Dynamic or interactive sites with **selenium** and **rvest**

Today, we'll turn our attention to **APIs**.

Packages for Today

First, a few packages for today:

```
pacman::p_load(lubridate, tidyverse, jsonlite, httr2, usethis,  
               fredr, XML, xml2, robotstxt, polite, rvest)
```

Also recommended: [**JSON Viewer**](#), a browser extension for Firefox and Chrome that renders JSON output nicely.

- Alternative: [**JSONView**](#) for both Firefox and Chrome

APIs

APIs

An **API (Application Programming Interface)** is a set of rules that carries out all this requesting, responding, and rendering. Some key concepts:

- **Server:** A powerful computer that runs an API.
- **Client:** A program that exchanges data with a server through an API.
- **Protocol:** The "etiquette" underlying how computers talk to each other (e.g. HTTP).
- **Methods:** The "verbs" that clients use to talk with a server.
 - The main one that we'll be using is **GET** (i.e. ask a server to **retrieve information**).
 - Other common methods are POST, PUT and DELETE.

APIs

An **API (Application Programming Interface)** is a set of rules that carries out all this requesting, responding, and rendering. Some key concepts:

- **Request:** What the client asks of the server (see Methods on last slide).
- **Response:** The server's response. This includes:
 - **Status Code** (e.g. "404" if not found, or "200" if successful).
 - **Header** (i.e. meta-information about the response).
 - **Body** (i.e. the actual contents that we're interested in).

With web APIs, we can access information *directly* from the API database if we can specify the correct URL, known as an **API endpoint**.

- [illegible]

API Endpoints

The reason the site looks like hot  is that its structured and formatted in a very particular way (that can be read directly into R!).

Usually these pages are formatted in one of two ways: **XML** or **JSON**

- We don't need too worry too much about the particular syntax of either

In general there are **three types** of API Endpoints:

1. **Explicit** API Endpoint, **no API Key**
2. **Explicit** API Endpoint, **API Key Required**
3. **Hidden** API Endpoints

Let's work through an example workflow for each. But first...

Some APIs that are free & legal

Some fun APIs:

- [TikTok \(unofficial\)](#)
- [Spotify](#)
- [YouTube](#)

This list of [Public APIs](#) has many more.

R API Packages

Many **R packages** have been built to make directly working with APIs easier¹. Some examples:

- **tidycensus** for American Community Survey microdata and other Census Bureau data
- **fredr** for more easily doing what we will do today
- **Quandl** for Nasdaq and other financial information
- **rtweet** for Twitter data
- **rnoaa** for NOAA weather data
- **prism** for PRISM gridded climate data (we'll work with this later on)
- **TADA** for Water Quality Portal Data (EPA, USGS)
- **rnassqs** for USDA's QuickStats API (agricultural data)
- **osmdata** for OpenStreetMap spatial data

¹ Note that a data acquisition project for this class requires you to engage *directly* with the API. Just using one of these packages doesn't count!

Explicit API Endpoint, No Key

Explicit API Endpoint, No Key

Let's work through an example of the first type of API: **explicit API without a key**.

Let's use the impressive **NYC Open Data** to look at... 🌲.

Seriously, trees.

Application 1: NYC Tree Census

First, open the [Tree Census page](#) in your browser (if you haven't done so already)

The screenshot shows the NYC OpenData website interface. At the top, there's a navigation bar with links for Home, Data, About, Learn, Alerts, and Contact Us, along with a search icon and a Sign In button. Below the navigation bar, there's a section for the '2015 Street Tree Census - Tree Data' dataset, categorized under 'Environment'. The dataset description mentions it's from the TreesCount! 2015 Street Tree Census, conducted by volunteers and staff organized by NYC Parks & Recreation and partner organizations. It includes tree species, diameter, and perception of health. A 'Read more' link is provided. To the right of the description, it shows 'Last Updated: May 9, 2022' and 'Data Provided By: Department of Parks and Recreation (DPR)'. Below the dataset description, there's a section titled 'About this Dataset' which contains a table of dataset information. The table includes fields for Updated (May 9, 2022), Data Last Updated (October 4, 2017), Metadata Last Updated (May 9, 2022), Date Created (June 3, 2016), Views (62.2K), and Downloads (17.7K). The 'Dataset Information' section includes Agency (Department of Parks and Recreation (DPR)), Update Frequency (Historical data), Automation (No), and Date Made Public (6/3/2016). The 'Attachments' section lists a file named 'StreetTreeCensus2015TreesDataDictionary20161102.pdf'.

NYC OpenData Home Data About ▾ Learn ▾ Alerts Contact Us | 🔍 Sign In

Actions ▾ Export

2015 Street Tree Census - Tree Data Environment

Street tree data from the TreesCount! 2015 Street Tree Census, conducted by volunteers and staff organized by NYC Parks & Recreation and partner organizations. Tree data collected includes tree species, diameter and perception of health. Accompanying blockface data is available indicating status of data collection and data release...

[Read more ▾](#)

Last Updated
May 9, 2022

Data Provided By
Department of Parks and Recreation (DPR)

About this Dataset

Updated May 9, 2022	Dataset Information
Data Last Updated October 4, 2017	Agency Department of Parks and Recreation (DPR)
Metadata Last Updated May 9, 2022	Update
Date Created June 3, 2016	Update Frequency Historical data
Views 62.2K	Automation No
Downloads 17.7K	Date Made Public 6/3/2016
	Attachments
	StreetTreeCensus2015TreesDataDictionary20161102.pdf

Application 1: NYC Tree Census

Click on **Actions > API** in upper-right to display the API Endpoint

The screenshot shows the NYC OpenData website interface. At the top, the 'NYC OpenData' logo is on the left, and navigation links (Home, Data, About, Learn, Alerts, Contact Us) and a search bar are on the right. A 'Sign In' button is also present. The main content area displays the dataset '2015 Street Tree Census - Tree Data' with a description, a 'Read more' link, and metadata including 'Last Updated: May 9, 2022' and 'Data Provided By: Department of Parks and Recreation (DPR)'. An 'Actions' dropdown menu is open, showing options: 'Query data', 'Visualize', 'API' (highlighted), 'Access via oData', and 'Share'. Below the dataset description, the 'About this Dataset' section provides further details: 'Updated: May 9, 2022', 'Data Last Updated: October 4, 2017', 'Metadata Last Updated: May 9, 2022', 'Date Created: June 3, 2016', 'Views: 62.2K', and 'Downloads: 17.7K'. The 'Dataset Information' table lists 'Agency: Department of Parks and Recreation (DPR)', 'Update Frequency: Historical data', 'Automation: No', and 'Date Made Public: 6/3/2016'. The 'Attachments' section lists a PDF file: 'StreetTreeCensus2015TreesDataDictionary20161102.pdf'.

NYC OpenData Home Data About ▾ Learn ▾ Alerts Contact Us

2015 Street Tree Census - Tree Data Environment

Street tree data from the TreesCount! 2015 Street Tree Census, conducted by volunteers and staff organized by NYC Parks & Recreation and partner organizations. Tree data collected includes tree species, diameter and perception of health. Accompanying blockface data is available indicating status of data collection and data release...

[Read more ▾](#)

Last Updated
May 9, 2022

Data Provided By
Department of Parks and Recreation (DPR)

Actions ▾

- Query data
Group, aggregate and more
- Visualize ▸
- API**
- Access via oData
- Share

About this Dataset

Updated
May 9, 2022

Data Last Updated: October 4, 2017 Metadata Last Updated: May 9, 2022

Date Created
June 3, 2016

Views: **62.2K** Downloads: **17.7K**

Dataset Information

Agency	Department of Parks and Recreation (DPR)
--------	--

Update

Update Frequency	Historical data
Automation	No
Date Made Public	6/3/2016

Attachments

- StreetTreeCensus2015TreesDataDictionary20161102.pdf

Application 1: NYC Tree Census

Click on **Actions > API** in upper-right to display the API Endpoint

The screenshot shows the NYC OpenData website interface. The top navigation bar includes links for Home, Data, About, Learn, Alerts, and Contact Us, along with a search icon and a Sign In button. On the right side of the page, there are buttons for 'Actions' and 'Export'. The main content area displays the '2015 Street Tree Census - Tree Data' dataset page. The dataset description mentions that the data includes tree species, diameter at breast height, and blockface data. The 'About this Dataset' section indicates the data was updated on May 9, 2022, and provides information on the last updated date (October 4, 2017) and the date created (June 3, 2016). The dataset has 62.2K views and 17.7K downloads. An 'Export dataset' modal is open, showing options to download the file or use the API endpoint. The modal also displays a warning about the default API limit being exceeded (1,000 rows) and provides a link to learn more. The API endpoint is shown as `https://data.cityofnewyork.us/resource/uvpi-gqnh.js`. The modal includes buttons for 'Cancel' and 'Copy to clipboard'.

NYC OpenData

Home Data About Learn Alerts Contact Us

Sign In

Actions Export

2015 Street Tree Census - Tree Data

Street tree data from the TreesCount project, collected and staff organized by NYC Parks and Recreation. The data includes tree species, diameter at breast height, and blockface data is available indicating the location of the tree.

Read more

About this Dataset

Updated
May 9, 2022

Data Last Updated: October 4, 2017
Metadata Last Updated: May 9, 2022

Date Created
June 3, 2016

Views: 62.2K
Downloads: 17.7K

Export dataset

2015 Street Tree Census - Tree Data

Download file API Endpoint

Data Format
JSON

All data (683788 rows)

Default API limit exceeded
Our API has a default limit of providing 1,000 rows. [Learn more](#) about how you can modify the default limit.

API Endpoint
`https://data.cityofnewyork.us/resource/uvpi-gqnh.js`

API documentation Developer portal

Cancel Copy to clipboard

StreetTreeCensus2015TreesDataDictionary20161102.pdf

Application 1: NYC Tree Census

Copy/Paste or click "go to" to open the API endpoint link in another tab.
You'll see a bunch of JSON text.



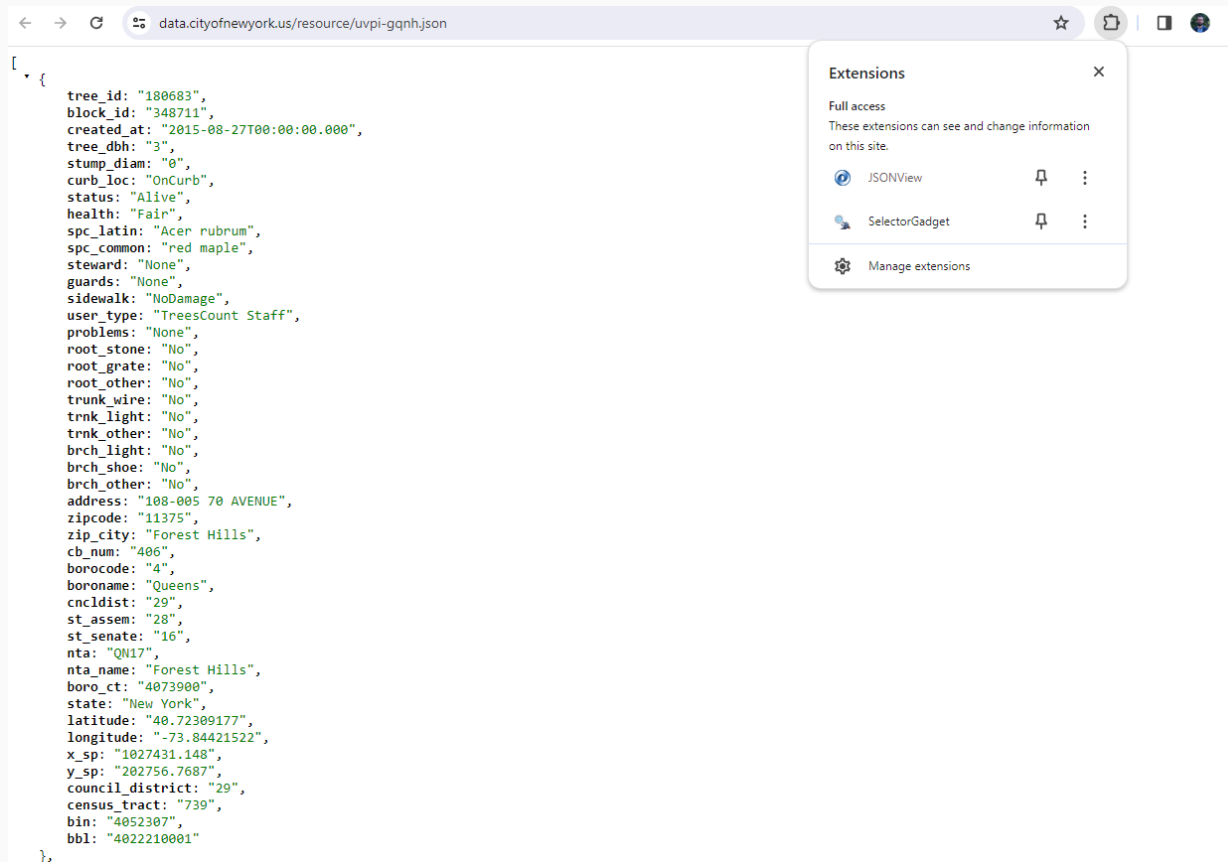
The screenshot shows a web browser window with the address bar displaying `data.cityofnewyork.us/resource/uvpi-gqnh.json`. The main content area shows a JSON array of tree records. Each record contains detailed information about a specific tree, including its ID, block ID, creation date, status, health, species, location, and various attributes like trunk diameter, height, and canopy spread. The records are formatted as JSON objects, with each object representing a single tree entry. The browser interface includes standard navigation buttons (back, forward, refresh) and a search bar.

```
{
  "tree_id": "180683",
  "block_id": "348711",
  "created_at": "2015-08-27T00:00:00.000",
  "tree_dbh": "3",
  "stump_diam": "0",
  "curb_loc": "OnCurb",
  "status": "Alive",
  "health": "Fair",
  "spc_latin": "Acer rubrum",
  "spc_common": "red maple",
  "steward": "None",
  "guards": "None",
  "sidewalk": "NoDamage",
  "user_type": "TreesCount Staff",
  "problems": "None",
  "root_stone": "No",
  "root_grate": "No",
  "root_other": "No",
  "trunk_wire": "No",
  "trnk_light": "No",
  "trnk_other": "No",
  "brch_light": "No",
  "brch_shoe": "No",
  "brch_other": "No",
  "address": "108-005 70 AVENUE",
  "zipcode": "11375",
  "zip_city": "Forest Hills",
  "cb_num": "406",
  "borocode": "4",
  "boroname": "Queens",
  "cncldist": "29",
  "st_assem": "28",
  "st_senate": "16",
  "nta": "QN17",
  "nta_name": "Forest Hills",
  "boro_ct": "4073900",
  "state": "New York",
  "latitude": "40.72309177",
  "longitude": "-73.84421522",
  "x_sp": "1027431.148",
  "y_sp": "202756.7687",
  "council_district": "29",
  "census_tract": "739",
  "bin": "4052307",
  "bbl": "4022 210001"
},
{
  "tree_id": "200540",
  "block_id": "315986",
  "created_at": "2015-09-03T00:00:00.000",
  "tree_dbh": "21",
  "stump_diam": "0",
  "curb_loc": "OnCurb",
  "status": "Alive",
  "health": "Fair",
  "spc_latin": "Quercus palustris",
  "spc_common": "pin oak",
  "steward": "None",
  "guards": "None",
  "sidewalk": "Damage",
  "user_type": "TreesCount Staff",
  "problems": "Stones",
  "root_stone": "Yes",
  "root_grate": "No",
  "root_other": "No",
  "trunk_wire": "No",
  "trnk_light": "No",
  "trnk_other": "No",
  "brch_light": "No",
  "brch_shoe": "No",
  "brch_other": "No",
  "address": "147-074 7 AVENUE",
  "zipcode": "11357",
  "zip_city": "Whitestone",
  "cb_num": "407",
  "borocode": "4",
  "boroname": "Queens",
  "cncldist": "19",
  "st_assem": "27",
  "st_senate": "11",
  "nta": "QN49",
  "nta_name": "Whitestone",
  "boro_ct": "4097300",
  "state": "New York",
  "latitude": "40.79411067",
  "longitude": "-73.81867946",
  "x_sp": "1034455.701",
  "y_sp": "228644.8374",
  "council_district": "19",
  "census_tract": "973",
  "bin": "4101931",
  "bbl": "4044 750045"
},
{
  "tree_id": "204026",
  "block_id": "218365",
  "created_at": "2015-09-05T00:00:00.000",
  "tree_dbh": "3",
  "stump_diam": "0",
  "curb_loc": "OnCurb",
  "status": "Alive",
  "health": "Good",
  "spc_latin": "Gleditsia triacanthos var. inermis",
  "spc_common": "honeylocust",
  "steward": "lor2",
  "guards": "None",
  "sidewalk": "Damage",
  "user_type": "Volunteer",
  "problems": "Stones",
  "root_stone": "No",
  "root_grate": "No",
  "root_other": "No",
  "trunk_wire": "No",
  "trnk_light": "No",
  "trnk_other": "No",
  "brch_light": "No",
  "brch_shoe": "No",
  "brch_other": "No",
  "address": "390 MORGAN AVENUE",
  "zipcode": "11211",
  "zip_city": "Brooklyn",
  "cb_num": "301",
  "borocode": "3",
  "boroname": "Brooklyn",
  "cncldist": "34",
  "st_assem": "50",
  "st_senate": "18",
  "nta": "BK90",
  "nta_name": "East Williamsburg",
  "boro_ct": "3044900",
  "state": "New York",
  "latitude": "40.71758074",
  "longitude": "-73.9366077",
  "x_sp": "1001822.831",
  "y_sp": "200716.8913",
  "council_district": "34",
  "census_tract": "449",
  "bin": "3338310",
  "bbl": "30288 70001"
},
{
  "tree_id": "204337",
  "block_id": "217969",
  "created_at": "2015-09-05T00:00:00.000",
  "tree_dbh": "10",
  "stump_diam": "0",
  "curb_loc": "OnCurb",
  "status": "Alive",
  "health": "Good",
  "spc_latin": "Gleditsia triacanthos var. inermis",
  "spc_common": "honeylocust",
  "steward": "None",
  "guards": "None",
  "sidewalk": "Damage",
  "user_type": "Volunteer",
  "problems": "Stones",
  "root_stone": "Yes",
  "root_grate": "No",
  "root_other": "No",
  "trunk_wire": "No",
  "trnk_light": "No",
  "trnk_other": "No",
  "brch_light": "No",
  "brch_shoe": "No",
  "brch_other": "No",
  "address": "1027 GRAND STREET",
  "zipcode": "11211",
  "zip_city": "Brooklyn",
  "cb_num": "301",
  "borocode": "3",
  "boroname": "Brooklyn",
  "cncldist": "34",
  "st_assem": "53",
  "st_senate": "18",
  "nta": "BK90",
  "nta_name": "East Williamsburg",
  "boro_ct": "3044900",
  "state": "New York",
  "latitude": "40.71353749",
  "longitude": "-73.93445616",
  "x_sp": "1002420.358",
  "y_sp": "199244.2531",
  "council_district": "34",
  "census_tract": "449",
  "bin": "3338342",
  "bbl": "3029 250001"
}
```

Application 1: NYC Tree Census

If you installed JSON Viewer/JSONView, it will look a lot nicer

- A lot like a transposed dataframe row...



Load JSON into R

Next, use the `fromJSON()` function in the [excellent jsonlite package](#) to read JSON into R.

```
nyc_trees <- fromJSON("https://data.cityofnewyork.us/resource/uvpi-  
gqnh.json") %>%  
  as.data.frame()  
nyc_trees
```

##	tree_id	block_id	created_at	tree_dbh	stump_diam
## 1	180683	348711	2015-08-27T00:00:00.000	3	0
## 2	200540	315986	2015-09-03T00:00:00.000	21	0
## 3	204026	218365	2015-09-05T00:00:00.000	3	0
## 4	204337	217969	2015-09-05T00:00:00.000	10	0
## 5	189565	223043	2015-08-30T00:00:00.000	21	0
## 6	190422	106099	2015-08-30T00:00:00.000	11	0
## 7	190426	106099	2015-08-30T00:00:00.000	11	0
## 8	208649	103940	2015-09-07T00:00:00.000	9	0
## 9	209610	407443	2015-09-08T00:00:00.000	6	0
## 10	192755	207508	2015-08-31T00:00:00.000	21	0
## 11	203719	302371	2015-09-05T00:00:00.000	11	0

Aside: API Limits

Note that while the full census of NYC trees contains over 700,000 trees, we only downloaded a small portion.

By default, the API limits us to 1,000 rows.

For our use case (seeing an example of the workflow) this isn't an issue, but [reading the documentation](#) reveals a way to get around that (if so desired).

Punchline: always read the API documentation!

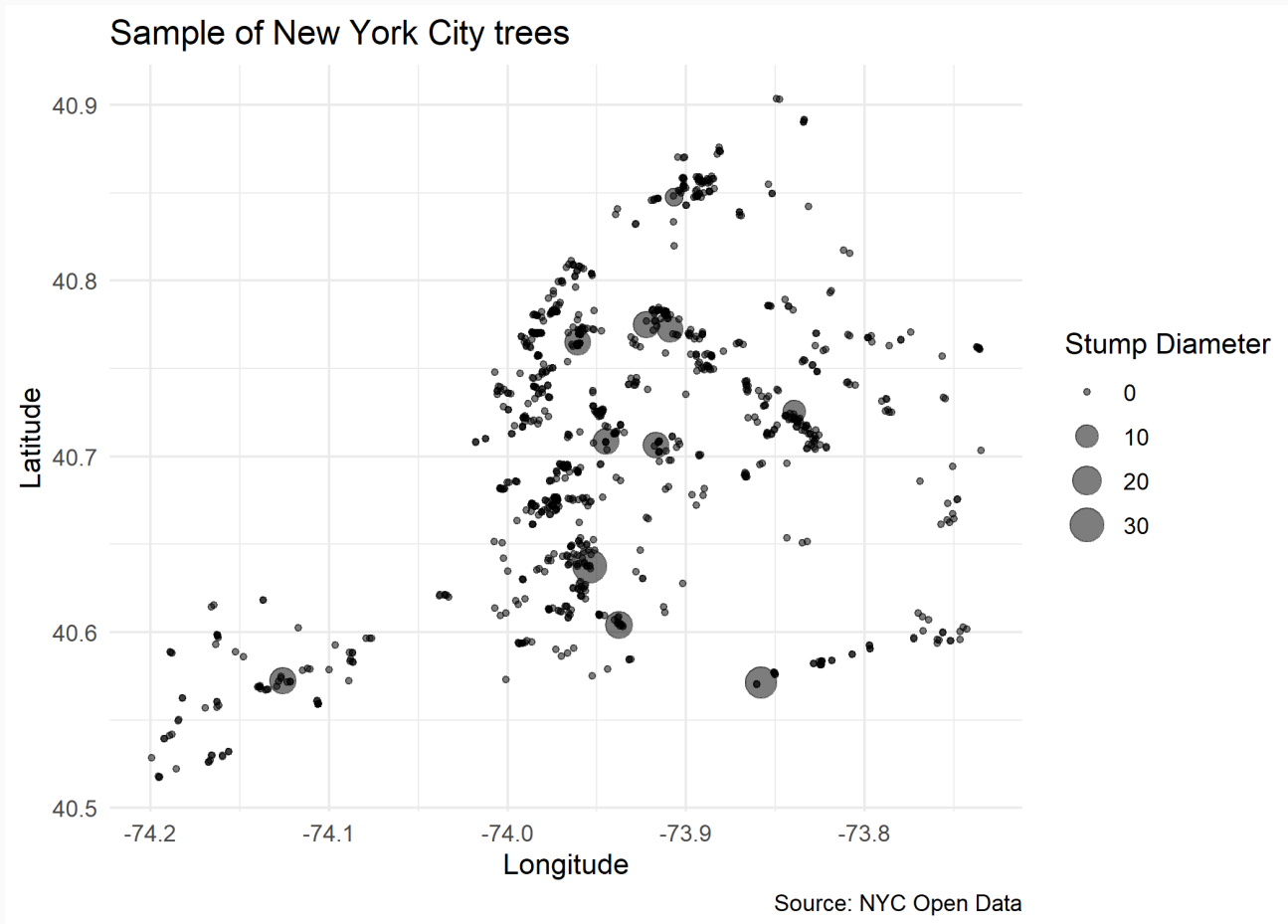
Work with the Data

Now we have the data, accessed through an API Endpoint!

Seriously, that's it. Now we can do whatever we want with the data - how about plotting it?

```
nyc_trees %>%
  select(longitude, latitude, stump_diam, spc_common, spc_latin, tree_id) %>%
  mutate_at(vars(longitude:stump_diam), as.numeric) %>%
  ggplot(aes(x=longitude, y=latitude, size=stump_diam)) +
  geom_point(alpha=0.5) +
  scale_size_continuous(name = "Stump Diameter") +
  labs(
    x = "Longitude", y = "Latitude",
    title = "Sample of New York City trees",
    caption = "Source: NYC Open Data"
  ) +
  theme_minimal()
```

Plotting Some Trees



Explicit API Endpoint with Key

Explicit API Endpoint with Key

For the second application, we're going to focus on the more common circumstance: an API endpoint that **requires an API key**.

An **API key** is a way to add a **unique ID signature** to every API call you make

- Required for all APIs linked to gov't agencies or institutions (e.g. Census, BEA).

Let's do an application working with the **Federal Reserve Bank of St. Louis (FRED) API**

Application 2: FRED API

FRED makes available a **ton of U.S. economic data** that can easily be accessed via API.

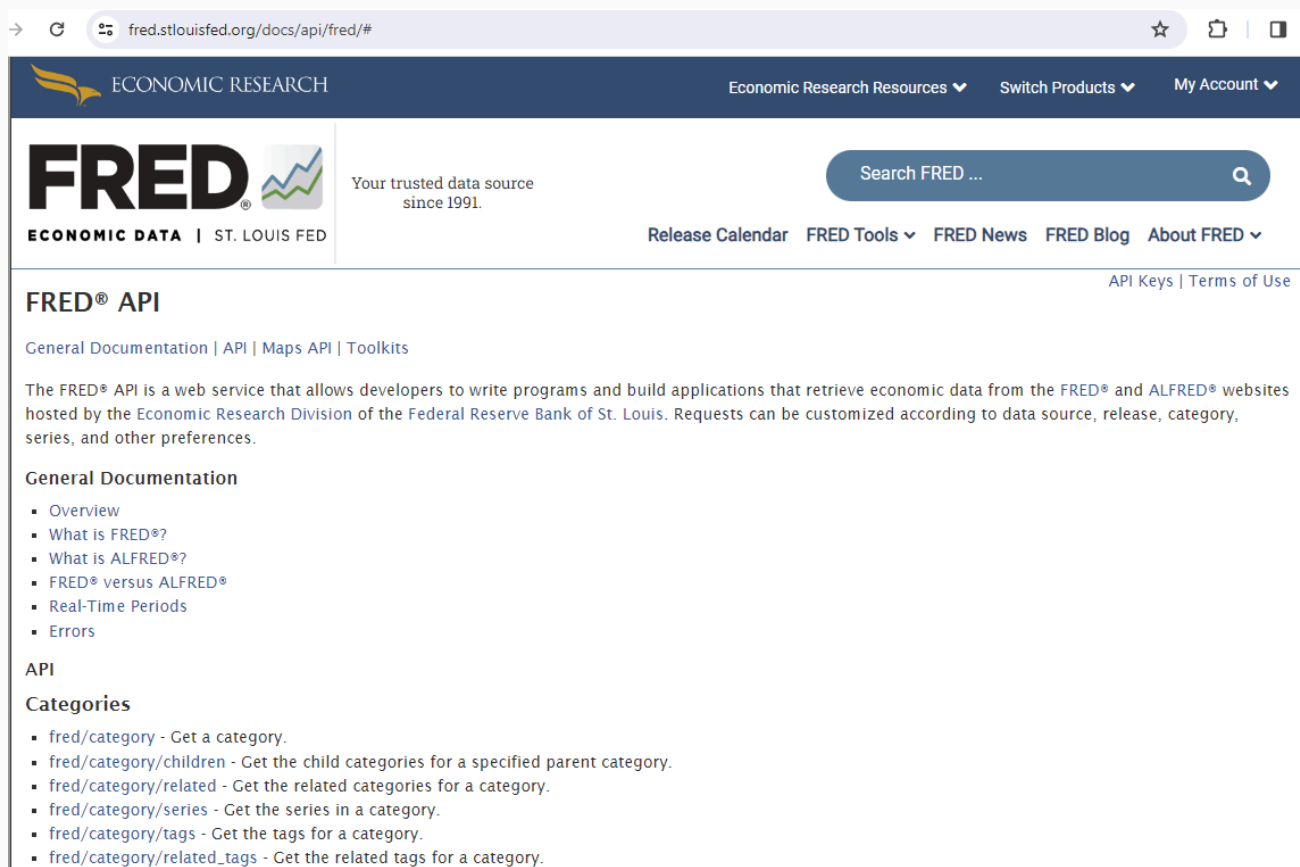


The only real difference from the last example: we first have to **register for an API key**

Application 2: FRED API

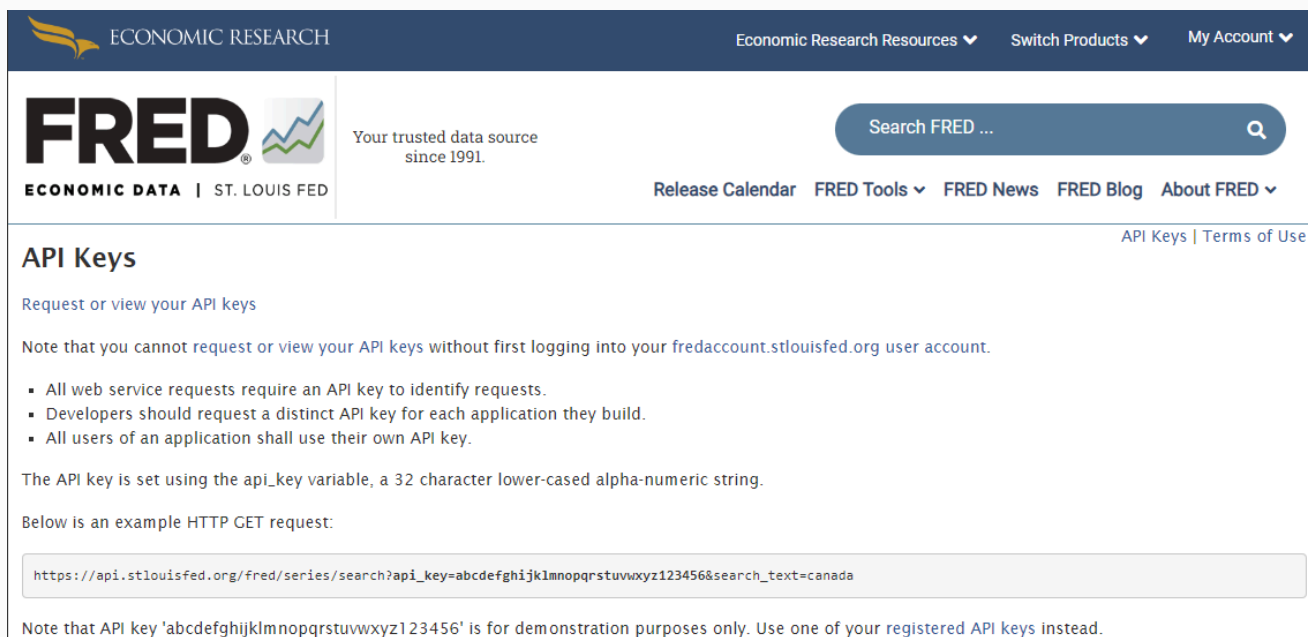
First, navigate to the [FRED API page](https://fred.stlouisfed.org/docs/api/fred/#)

```
knitr::include_graphics("images/fred_1.png")
```

A screenshot of the FRED API documentation page. The browser address bar shows 'fred.stlouisfed.org/docs/api/fred/#'. The page header includes the FRED logo, 'ECONOMIC DATA | ST. LOUIS FED', and navigation links like 'Economic Research Resources', 'Switch Products', and 'My Account'. A search bar is present with the text 'Search FRED ...'. The main content area is titled 'FRED® API' and includes links for 'General Documentation | API | Maps API | Toolkits'. A paragraph describes the FRED API as a web service for retrieving economic data. Below this, there are sections for 'General Documentation' (with links to Overview, What is FRED®, What is ALFRED®, FRED® versus ALFRED®, Real-Time Periods, and Errors) and 'API Categories' (with links to fred/category, fred/category/children, fred/category/related, fred/category/series, fred/category/tags, and fred/category/related_tags).

Application 2: FRED API

Click on "API Keys" in the upper-right then "Request or view your API keys"



The screenshot shows the FRED (Federal Reserve Economic Data) website. The header includes the FRED logo, the text "ECONOMIC DATA | ST. LOUIS FED", and navigation links for "Economic Research Resources", "Switch Products", and "My Account". A search bar is also present. The main content area is titled "API Keys" and includes a link to "Request or view your API keys". It contains a note about logging into a user account and a list of requirements for API keys. An example HTTP GET request is provided in a code block.

ECONOMIC RESEARCH Economic Research Resources ▼ Switch Products ▼ My Account ▼

FRED Your trusted data source since 1991.

ECONOMIC DATA | ST. LOUIS FED Release Calendar FRED Tools ▼ FRED News FRED Blog About FRED ▼

API Keys

[Request or view your API keys](#)

Note that you cannot request or view your API keys without first logging into your fredaccount.stlouisfed.org user account.

- All web service requests require an API key to identify requests.
- Developers should request a distinct API key for each application they build.
- All users of an application shall use their own API key.

The API key is set using the `api_key` variable, a 32 character lower-cased alpha-numeric string.

Below is an example HTTP GET request:

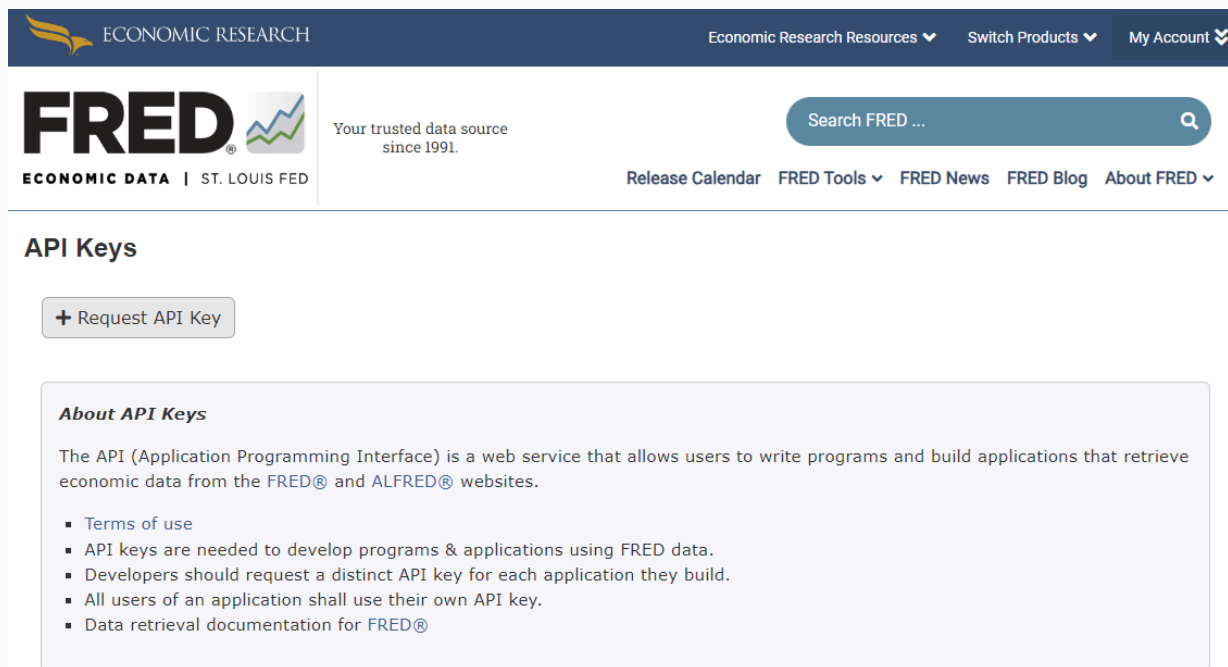
```
https://api.stlouisfed.org/fred/series/search?api_key=abcdefghijklmnopqrstuvwxyz123456&search_text=canada
```

Note that API key 'abcdefghijklmnopqrstuvwxyz123456' is for demonstration purposes only. Use one of your [registered API keys](#) instead.

Application 2: FRED API

Make an account, then request an API Key

- My Account > API Keys > Request API Key



The screenshot shows the FRED (Federal Reserve Economic Data) website's 'API Keys' page. The header is dark blue with 'ECONOMIC RESEARCH' on the left and navigation links for 'Economic Research Resources', 'Switch Products', and 'My Account' on the right. Below the header, the FRED logo is prominent, accompanied by the tagline 'Your trusted data source since 1991.' and 'ECONOMIC DATA | ST. LOUIS FED'. A search bar is located to the right of the logo. Below the header, the page title 'API Keys' is displayed. A button labeled '+ Request API Key' is visible. A section titled 'About API Keys' provides information about the API service, stating it is a web service for retrieving economic data. It includes a list of terms of use: API keys are needed for programs and applications; developers should request a distinct API key for each application; all users must use their own API key; and there is data retrieval documentation for FRED.

ECONOMIC RESEARCH

Economic Research Resources ▼ Switch Products ▼ My Account ▼

FRED Your trusted data source since 1991.

ECONOMIC DATA | ST. LOUIS FED

Search FRED ...

Release Calendar FRED Tools ▼ FRED News FRED Blog About FRED ▼

API Keys

+ Request API Key

About API Keys

The API (Application Programming Interface) is a web service that allows users to write programs and build applications that retrieve economic data from the FRED® and ALFRED® websites.

- [Terms of use](#)
- API keys are needed to develop programs & applications using FRED data.
- Developers should request a distinct API key for each application they build.
- All users of an application shall use their own API key.
- Data retrieval documentation for FRED®

Setting API Keys in R

Treat your API keys **like passwords**

- You're responsible for following API terms & conditions, and you can't control that if someone else is using your key

This means we shouldn't ever **hardcode API keys into R scripts**.

Instead, we can define them as **environment variables**, either

1. For the current R session only
2. Globally across all R sessions

Environment Variables: Current Session

To define an environment variable for **your current session only**, run the following **in your console**:²

```
Sys.setenv(FRED_API_KEY = "YOUR_KEY_HERE")
```

Replacing the fake API key with your actual key.

You can then use an environment variable in your script with

```
# assign to R object and print out  
my_key ← Sys.getenv("FRED_API_KEY")
```

² setting `Sys.setenv()` in scripts defeats the entire purpose!

Environment Variables: Globally

To set an environment variable **globally across all sessions**, we'll need to add a special `~/.Renviron` file.

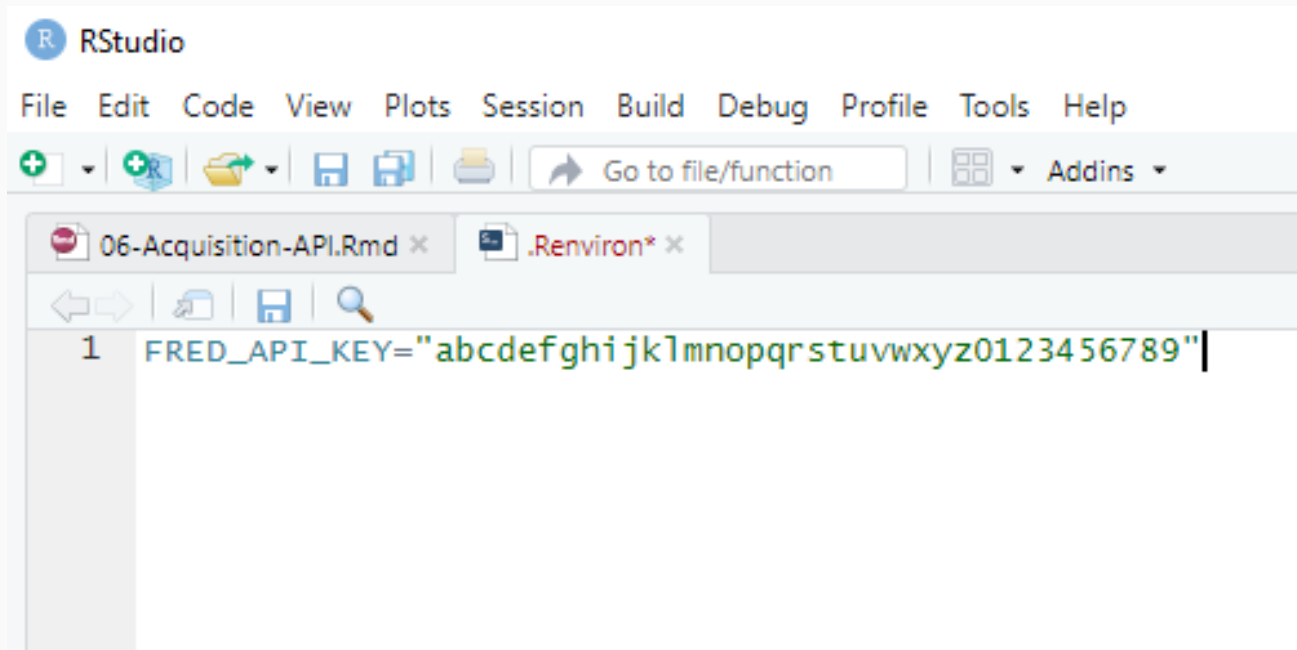
- Text file that lives in your home directory (hence the `~/` path)

To edit, we can use the ``usethis::edit_r_environ()` function:

```
usethis::edit_r_environ()
```

Environment Variables: Globally

This opens your `~/Renvi` file in a new tab, where you can easily add a line defining your API key.



Hit save, then restart R or refresh your environment session to use it:

```
## Only necessary when reading in a new Renvi variable  
readRenvi("~/Renvi")
```

Using the FRED API

With your API key set, let's figure out how to use the FRED API.

Looking at the [FRED API documentation](#) you'll see

- There are many types of requests you can make of FRED using the API
- If we want the underlying data for a data series, we want the [fred/series/observations](#) path.

Using the FRED API

Clicking the link takes us to [specific documentation for that path](#)

Here we can learn

1. In which **file formats** the API will return data
 - XML, JSON, zipped tab delimited, zipped Excel
2. The exact **syntax** for making a request
 - Potentially specific to each format
3. The **parameters** that you can provide
 - e.g. function arguments

Using the FRED API

One required parameter is `series_id`.

- Let's look at the median home listing price in East Lansing.
- By searching the FRED website, I found that the **ID** for this series is `MELIPRMSA29620`.

For the `file_type`, you can use either XML or JSON.

- Both can easily be read into R.
- Since we already saw JSON, let's try using XML.

Try out the API right in your browser. Paste this into your location bar, replacing the fake API key with your own:

```
https://api.stlouisfed.org/fred/series/observations?  
series_id=MELIPRMSA29620&api_key=YOURKEYHERE&file_type=json
```

Using the FRED API

To use the API in R, we'll use the tidyverse package **httr2** ("hitter 2"). This is an updated version of **httr** that adds more modern features and is pipeable!

- I find it much more intuitive

First we need to **create a request** with `request()`

```
req ← request("https://api.stlouisfed.org/")
```

httr2: Create Request

```
req %>% req_dry_run()
```

```
## GET / HTTP/1.1  
## Host: api.stlouisfed.org  
## User-Agent: httr2/1.1.0 r-curl/6.2.0 libcurl/8.10.1  
## Accept: */*  
## Accept-Encoding: deflate, gzip
```

Here we see that we've initiated a request of type **GET**. A **GET** method lets us "get" pre-existing data from a specific API endpoint

The alternative is a **POST** where we **submit data that results in creation of a new resource**

- **httr2** will automatically shift to **POST** if we add contents to our request!

httr2: Create Request

```
## GET / HTTP/1.1
## Host: api.stlouisfed.org
## User-Agent: httr2/1.1.0 r-curl/6.2.0 libcurl/8.10.1
## Accept: */*
## Accept-Encoding: deflate, gzip
```

We also see 4 other useful bits of information that represent the **HTTP headers**

- We can modify these as needed later

httr2: Modify Request

Once we have our request, we can **modify it** with the `req_` functions:

Function	Task
<code>req_body_(raw/file/json/json_modify/form/multipart</code>	send data in the request body (for a POST)
<code>req_(headers/headers_redacted</code>	modify the request headers
<code>req_(url/url_relative/url_query/url_path/url_path_append()</code>	modify the request URL
<code>req_progress()</code>	Add a progress bar (for long requests)
<code>req_timeout()</code>	Set time limit for a request
<code>req_user_agent()</code>	Sets the user-agent of the request
<code>req_proxy()</code>	Send the request through a proxy server
<code>req_cookie_(preserve/set)</code>	set and preserve cookies
<code>req_method()</code>	Set request to less common methods (HEAD, DELETE, PATCH, UPDATE, OPTIONS)

httr2: Modify Request

Let's start by telling the request the endpoint we want to query (`fred/series/observations`).

- Since we're appending to the FRED API url, we need `req_url_relative()`

```
req ← req_url_relative(req, "fred/series/observations")  
req_dry_run(req)
```

```
## GET /fred/series/observations HTTP/1.1  
## Host: api.stlouisfed.org  
## User-Agent: httr2/1.1.0 r-curl/6.2.0 libcurl/8.10.1  
## Accept: */*  
## Accept-Encoding: deflate, gzip
```

httr2: Modify Request

Now we need to add in some of our **query parameters**.

To do this, we need to follow the **syntax of the API documentation**:

```
?series_id=MELIPRMSA29620&api_key=YOURKEYHERE&file_type=json
```

Here we have **three parameters**:

- `series_id`: which series we want to get
- `api_key`: our personal API key
- `file_type`: the format we want the file in

httr2: Modify Request

We can add our **query parameters** using `req_url_query()`:

- Note: the parameter names are wrapped in backticks!

```
req <- req_url_query(req,  
  series_id = "MELIPRMSA29620",  
  api_key = my_key,  
  file_type = "xml")  
req_dry_run(req)
```

```
## GET /fred/series/observations?  
series_id=MELIPRMSA29620&api_key=84236ae7dcd2141f2e4efea249f503c1&file_type=xml  
HTTP/1.1  
## Host: api.stlouisfed.org  
## User-Agent: httr2/1.1.0 r-curl/6.2.0 libcurl/8.10.1  
## Accept: */*  
## Accept-Encoding: deflate, gzip
```

httr2: Make the Request

Once your request URL is set, use `req_perform()` to **perform the request!**

```
resp ← req_perform(req)
```

http2: Looking at the Response

GET

[https://api.stlouisfed.org/fred/series/observations?
series_id=MELIPRMSA29620&api_key=84236ae7dcd2141f2e4efea249f503](https://api.stlouisfed.org/fred/series/observations?series_id=MELIPRMSA29620&api_key=84236ae7dcd2141f2e4efea249f503)

Status: 200 OK

Content-Type: text/xml

Body: In memory (12020 bytes)

Looking at our response, we see that it worked (Status: 200 OK)!

httr2: Looking at the Response

What we want to do now is extract the body, which we can do with one of the `resp_body_` functions

Function	Task
<code>resp_body_raw</code>	return the raw bytes
<code>resp_body_string</code>	returns a UTF-8 string
<code>resp_body_json</code>	returns parsed JSON
<code>resp_body_html</code>	returns parsed HTML
<code>resp_body_xml</code>	returns parsed XML
<code>resp_has_body</code>	logical check if the response has a body

httr2: Looking at the Response

We requested XML, so let's use `resp_body_xml()` to parse it:

```
resp_body <- resp_body_xml(resp)
```

httr2: Looking at the Response

This gets us the XML document, which

- Starts with information about the data (start and end dates, order and sort, number of results, etc.)
- Contains each observation as a nested entry

```
resp_body
```

```
## {xml_document}  
## <observations realtime_start="2025-10-22" realtime_end="2025-10-22"  
observation_start="1600-01-01" observation_end="9999-12-31" units="lin"  
output_type="1" file_type="xml" order_by="observation_date"  
sort_order="asc" count="111" offset="0" limit="100000">  
## [1] <observation realtime_start="2025-10-22" realtime_end="2025-10-22"  
date= ...  
## [2] <observation realtime_start="2025-10-22" realtime_end="2025-10-22"  
date= ...  
## [3] <observation realtime_start="2025-10-22" realtime_end="2025-10-22"  
date= ...  
## [4] <observation realtime_start="2025-10-22" realtime_end="2025-10-22"
```

XML

See the XML structure with `xml_structure()`:

```
xml_structure(resp_body)
```

```
## <observations [realtime_start, realtime_end, observation_start,  
observation_end, units, output_type, file_type, order_by, sort_order,  
count, offset, limit]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

```
##   <observation [realtime_start, realtime_end, date, value]>
```

XML

This shows us the nested structure of our body, where each row is stored in a `<observation >` element.

We can retrieve these all using `xml_find_all()` and the `xpath` argument!

- Spoiler: it turns out that `xpaths` are actually the syntax for navigating an XML document

```
xml_find_all(resp_body, xpath = "//observation")

## {xml_nodeset (111)}
## [1] <observation realtime_start="2025-10-22" realtime_end="2025-10-22"
date= ...
## [2] <observation realtime_start="2025-10-22" realtime_end="2025-10-22"
date= ...
## [3] <observation realtime_start="2025-10-22" realtime_end="2025-10-22"
date= ...
## [4] <observation realtime_start="2025-10-22" realtime_end="2025-10-22"
date= ...
```

XML

```
{xml_nodeset (111)}
```

```
[1] <observation realtime_start="2025-10-22" realtime_end="2025-10-22" d ...
```

```
[2] <observation realtime_start="2025-10-22" realtime_end="2025-10-22" d ...
```

`xml_find_all()` yields an xml nodeset, where each node contains four attributes: `realtime_start`, `realtime_end`, `date`, and `value`

- `xml_attrs()` to see the attributes

```
xml_find_all(resp_body, xpath = "//observation") %>%  
xml_attrs()
```

```
## [[1]]
```

```
## realtime_start    realtime_end          date          value
```

```
## "2025-10-22"      "2025-10-22"      "2016-07-01"      " "
```

XML

To extract a specific attribute's values, use `xml_attr("attribute_name")`

For example, to get the `realtime_start` values:

```
xml_find_all(resp_body, xpath = "//observation") %>%  
xml_attr("realtime_start") %>% head()
```

```
## [1] "2025-10-22" "2025-10-22" "2025-10-22" "2025-10-22" "2025-10-22"  
## [6] "2025-10-22"
```

XML

And we can build our dataframe!

```
resp_df <- data.frame(  
  date = xml_find_all(resp_body, xpath = "//observation") %>%  
xml_attr("date") %>% ymd(),  
  median_price = xml_find_all(resp_body, xpath = "//observation") %>%  
xml_attr("value") %>% as.numeric()  
)
```

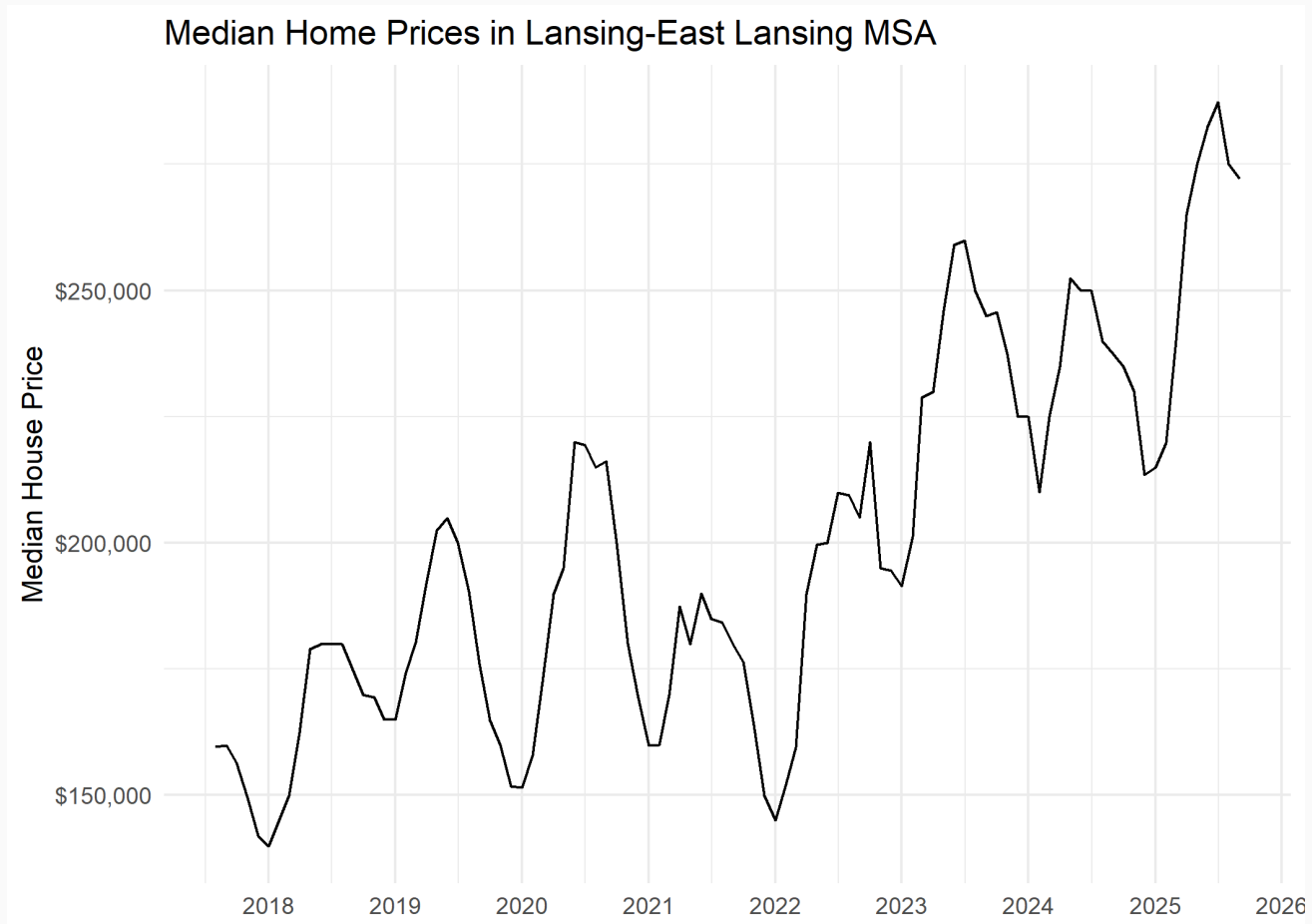
Using the FRED API

Plotting:

```
drop_na(resp_df) %>%  
ggplot(aes(x = date, y = median_price)) +  
  geom_line() +  
  theme_minimal() +  
  scale_y_continuous(labels = scales::label_dollar(),  
                      name = "Median House Price") +  
  scale_x_date(name = NULL,  
               date_breaks = "1 year",  
               date_labels = "%Y") +  
  labs(title = "Median Home Prices in Lansing-East Lansing MSA")
```

Using the FRED API

Plotting:



Challenge

How has East Lansing's pandemic housing market compared with other cities?

- Get similar data from 2-3 other housing markets you're interested in.
- Plot time-series graphs for each housing market.
- More advanced: combine all the results in 1 dataframe (keep it tidy!).

Hidden APIs

Hidden APIs

The third kind of APIs are known as **Hidden APIs**.

These are endpoints used for a website's **Internal page generation** and is not intended for public use.

- For example, Airbnb uses or has used a hidden API to display search results on its web interface.
- Grant McDermott has an [example](#) of how to find and use hidden APIs.
- We're not going to go in-depth into hidden API access because...

Hidden APIs

One caveat: the legality of using hidden APIs is **less clear**

- In April 2022 (*HiQ Labs v. LinkedIn*), the Ninth Circuit Court of Appeals held that webscraping is legal in the U.S. so long as the data is **publicly available**
- Do hidden APIs constitute publicly available information?

Even if generally legal, web scraping may be a violation of a website's Terms & Conditions

- Airbnb: "Do not use bots, crawlers, scrapers, or other automated means to access or collect data or other content from or otherwise interact with the Airbnb Platform."
- Your IP could be blocked or your account could be blocked/blacklisted

Considerate Web Scraping

Considerate Web Scraping

In addition to our internal evaluation of

Just because we can, doesn't mean we should

another useful tool in this process is **checking for and reading the robots.txt file**.

Many sites have a `robots.txt` file that outlines what parts of the site web crawlers *can* and *can't* access.

- Specific pages or API endpoints
- Specific types of users
- Speeds at which you can scrape

Considerate Web Scraping

The **robotstxt** package makes it easy to check for and read these files. For example, Wikipedia:

```
get_robotstxt(domain = "https://www.wikipedia.org")

## [robots.txt]
## -----
##
## # robots.txt for http://www.wikipedia.org/ and friends
## #
## # Please note: There are a lot of pages on this site, and there are
## # some misbehaved spiders out there that go _way_ too fast. If you're
## # irresponsible, your access to the site may be blocked.
## #
##
## # Observed spamming large amounts of https://en.wikipedia.org/?curid=NNNNNN
## # and ignoring 429 ratelimit responses, claims to respect robots:
## # http://mj12bot.com/
## User-agent: MJ12bot
## Disallow: /
##
## # advertising-related bots:
## User-agent: Mediapartners-Google*
## Disallow: /
##
## # Wikipedia work bots:
```

Considerate Web Scraping

Another great R package is **polite**, which builds upon **rvest** to provide a workflow that both queries the `robots.txt` file and performs the scraping itself.

Two main functions/steps:

bow()

- Introduces the client to the host
- Asks permission to web scrape
 - Inquires against the `robots.txt`

scrape()

- Retrieves data from the remote server

Let's see an example by getting [Des Linden's 2024 NYC Marathon Results](#).



Des Linden Finishes 11th at the 2024 New York City Marathon

She was the fourth American and the second masters runner.

BY [SARAH LORGE BUTLER](#) PUBLISHED: NOV 03, 2024 12:25 PM EST

 **SAVE ARTICLE**



polite

Start by introducing yourself with `bow()`:

```
bow("https://www.runnersworld.com/races-places/a62786311/des-linden-11th-new-york-city-marathon-2024/", verbose = T)
```

```
## <polite session> https://www.runnersworld.com/races-places/a62786311/des-linden-11th-new-york-city-marathon-2024/  
##      User-agent: polite R package  
##      robots.txt: 127 rules are defined for 5 bots  
##      Crawl delay: 5 sec  
##      The path is scrapable for this user-agent
```

polite

```
## <polite session> https://www.runnersworld.com/races-  
places/a62786311/des-linden-11th-new-york-city-marathon-2024/  
##      User-agent: polite R package  
##      robots.txt: 117 rules are defined for 10 bots  
##      Crawl delay: 5 sec  
##      The path is scrapable for this user-agent
```

`bow()` creates a **polite session** and tells us what's allowed given the rules outlined in the `robots.txt`.

- Here, we start with a 5 sec delay.
 - To change the time, use `set_scrape_delay(time)`

polite

Now, let's try and scrape the results page using `scrape()`. Arguments:

- `bow`: the polite session
- `query`: anything you want to append to the url
- `accept`: what type of data to expect (e.g. HTML, JSON, CSV)
- `verbose`: extra feedback

```
rw_session ← bow("https://www.runnersworld.com/races-  
places/a62786311/des-linden-11th-new-york-city-marathon-2024/", verbose  
= T)
```

```
rw_des_24 ← scrape(rw_session, accept = "html")
```

polite robots.txt

We can view the robots file directly from our session as the `$robotstxt` object:

```
rw_session$robotstxt
```

```
## $domain
## [1] "https://www.runnersworld.com"
##
## $text
## [1] "\nUser-agent: CCBot\nDisallow: /\n\nUser-agent: ChatGPT-User\nUser-agent:
GPTBot\nUser-agent: OAI-SearchBot\nDisallow: /au/\nDisallow: /cn/\nDisallow: /dk/\n\n[ ... 97
lines omitted ... ]"
##
## $robexclobj
## <Robots Exclusion Protocol Object>
## $bots
## [1] "CCBot"          "ChatGPT-User"  "GPTBot"        "OAI-SearchBot"
## [5] "*"
##
## $comments
## [1] line      comment
## <0 rows> (or 0-length row.names)
##
## $permissions
##
```

polite

What did we get back?

```
rw_des_24
```

```
## {html_document}  
## <html lang="en-US">  
## [1] <head>\n<meta http-equiv="Content-Type" content="text/html;  
charset=UTF-8 ...  
## [2] <body>\n<template id="page-settings" data-nav-height="126">  
</template><di ...
```

A: an HTML document, that we know how to interact with!

polite

Using **rvest** methods to retrieve the text:

```
pars <- html_elements(rw_des_24, ".article-body-content") %>%  
  html_elements("p")  
pars[[1]] %>%  
  html_text2() %>%  
  str_extract("[0-9]:[0-9]+:[0-9]+")
```

```
## [1] "2:29:32"
```

Considerate Web Scraping

Best Practices for considerate web scraping:

- Check the `robots.txt` file and scrape what's allowed
- If no `robots.txt` file exists, stick to
 1. Public information
 2. Scraped at a reasonable speed (i.e. human frequency)
- Scrape the data **once**, not **every time you run the script**
- Ask yourself: "would I feel comfortable having an undergrad RA scrape this?"

Table of Contents

1. **Prologue**
2. **APIs**
3. **Explicit API Endpoint, No Key**
4. **Explicit API Endpoint with Key**
5. **Hidden APIs**
6. **Considerate Web Scraping**